

NWCTL v0.2.2 User Manual

nwctl User Manual

Nuway Autoware Command Toolkit — Multi-user Autoware Docker testing and development toolkit

Quick Command Reference

User Management

Command	Description
<code>sudo nwctl register <name> --src <path></code>	Admin registers a target with an automatic development DOMAIN
<code>sudo nwctl set-domain <name> --domain-id <10-232></code>	Admin assigns a development/integration DOMAIN
<code>sudo nwctl set-domain <name> --domain-id <0-9> --production</code>	Admin authorizes a vehicle/production DOMAIN
<code>sudo nwctl update <name> --src <path></code>	Update user's source path
<code>sudo nwctl unregister <name></code>	Remove user and delete build/install/log workspace
<code>sudo nwctl unregister <name> --keep-workspace</code>	Remove user (explicitly keep workspace)
<code>nwctl list</code>	List all registered users and their DOMAIN_IDs

Run Modes

Command	Description
<code>nwctl <name> planning-sim</code>	Launch planning simulation (uses prebuilt image)
<code>nwctl <name> rosbag-replay</code>	Launch rosbag replay (uses prebuilt image)
<code>nwctl <name> shell</code>	Enter development shell (mounts user source)
<code>nwctl <name> stop</code>	Stop all running containers for a user
<code>nwctl <name> clean</code>	Clean user's build cache

Status & Maintenance

Command	Description
<code>nwctl status</code>	Show all running aw-* containers
<code>nwctl disk</code>	Show disk usage per user (workspace)
<code>nwctl cleanup</code>	Remove orphan containers and temporary files
<code>nwctl cleanup --dry-run</code>	Preview what cleanup would remove (no changes)
<code>nwctl check-env [mode]</code>	Pre-flight check (Docker/GPU/image/display/data/resources)
<code>nwctl version</code>	Show version number
<code>nwctl pull</code>	Pull or update the configured container image
<code>nwctl pull --profile cpu</code>	Pull the no-GPU Humble development image
<code>nwctl completion <bash\ zsh></code>	Print completion setup for a source checkout
<code>nwctl -h</code>	Show help

CPU and NVIDIA Profiles

The default auto profile selects `nvidia` only when both an NVIDIA GPU and the Docker NVIDIA runtime are available; otherwise it selects `cpu`.

Profile	Image	Runtime
<code>cpu</code>	<code>ghcr.io/autowarefoundation/autoware:universe-devel-humble</code>	Software rendering, no NVIDIA runtime
<code>nvidia</code>	<code>ghcr.io/autowarefoundation/autoware:universe-devel-cuda</code>	<code>--runtime=nvidia</code>

`nwctl check-env shell --profile cpu`

```
nwctl <name> shell --profile cpu
nwctl <name> planning-sim --profile cpu
```

Set NWCTL_PROFILE=cpu to make CPU mode the default for a host/session.

Run Modes — Details

1. planning-sim — Planning Simulation

```
nwctl <name> planning-sim
```

- Uses /opt/autoware inside the image (prebuilt) — **no local compilation required**
- Opens an rviz2 window via X11 forwarding automatically

Steps:

1. Wait for the rviz2 window to appear (~30 seconds)
2. Click **2D Pose Estimate** in the toolbar, then click and drag on the map to set the initial pose
3. Click **2D Goal Pose** to set the destination
4. Click the buttons in the **AutowareStatePanel** to start autonomous driving

2. rosbag-replay — Bag File Replay

```
nwctl <name> rosbag-replay
nwctl <name> rosbag-replay --bag <subdirectory> --rate 0.5
```

- Uses the prebuilt version inside the image
- Starts Autoware and runs `ros2 bag play` after the initialization delay
- `--bag` selects a subdirectory below `--rosbag-path`; `--rate` controls speed
- The CPU profile disables the CUDA/ML-model perception pipeline and waits 60 seconds before playback; map loading, sensor decoding, localization, and rviz remain enabled
- Set `NWCTL_ROSBAG_START_DELAY` to override the automatic playback delay
- In the rviz **Views** panel, set **Target Frame** to `base_link` to follow the vehicle

3. shell — Development Mode

```
nwctl <name> shell
```

- Mounts the user's source directory to `/workspace/src` inside the container
- Overlays `/opt/autoware` (prebuilt) as the base dependency layer

Common operations inside the container:

```
# Incremental build – single package (most common)
colcon build --packages-select <pkg> --symlink-install
```

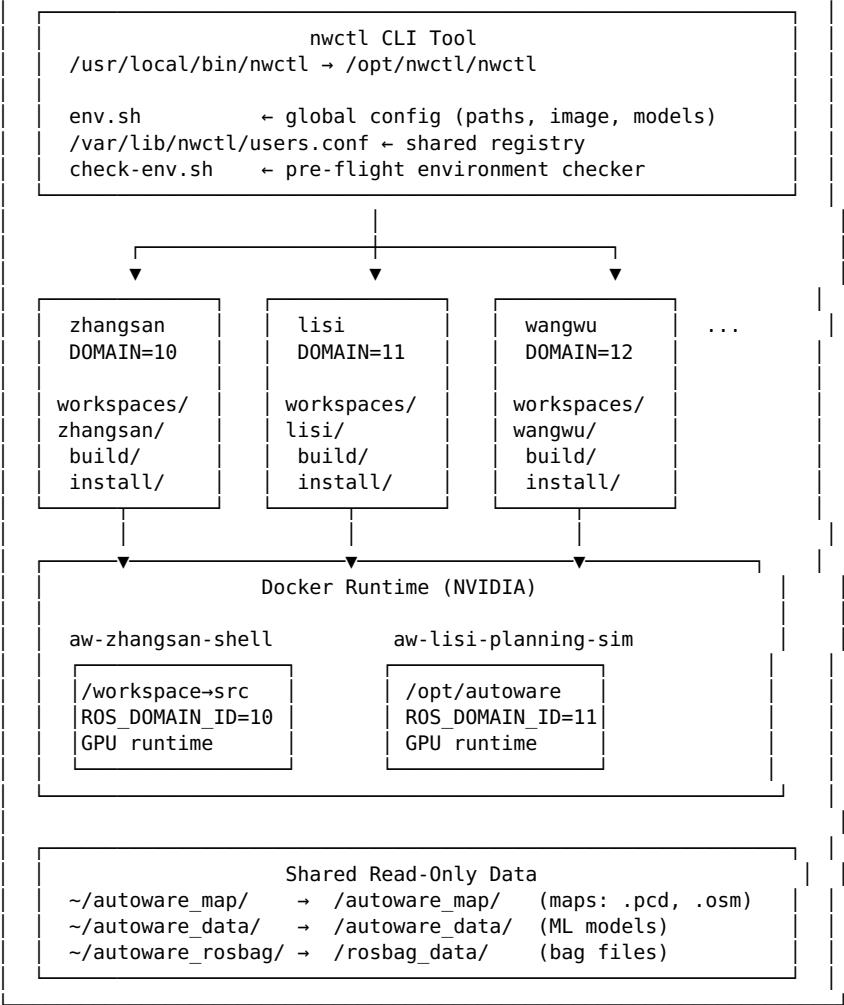
```
# Build a package with all its dependencies
colcon build --packages-up-to <pkg> --symlink-install
```

```
# Full build (first time only – takes a while)
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release
```

```
# Re-source and test with planning simulation
source /workspace/install/setup.bash
ros2 launch autoware_launch planning_simulator.launch.xml \
  map_path:=/autoware_map/sample-map-rosbag \
  vehicle_model:=sample_vehicle \
  sensor_model:=sample_sensor_kit
```

Architecture Diagram

Host Machine (Jetson Orin)



Container Mode Comparison

Mode	Source	setup.bash	Use Case
planning-sim	Prebuilt in image	/opt/autoware/setup.bash	Testing, demo, validation
rosbag-replay	Prebuilt in image	/opt/autoware/setup.bash	Data replay and analysis
shell	User-mounted source	/workspace/install/setup.bash	Development, debugging, algorithm changes

Isolation Mechanisms

Resource	Method	Details
ROS topics	ROS_DOMAIN_ID	Development IDs 10-199 are auto-assigned; integration 200-232 and vehicle/production 0-9 are admin-assigned
Source code	Separate git clones	Each user manages their own repository and branches
Build artifacts	Separate mounts	/var/lib/nwctl/workspaces/<user>/build, install, log
Container names	aw-<user>-<mode>	Prevents conflicts, easy to identify
Registry writes	flock (10s timeout)	users.conf.lock prevents concurrent write conflicts
GPU	Shared	NVIDIA runtime supports multiple containers simultaneously

Typical Workflow

Admin (one-time setup)

```
# Pull the Docker image
docker pull ghcr.io/autowarefoundation/autoware:universe-devel-cuda

# Install nwctl
cd ~/autoware/nwctl
sudo ./install.sh
# Bash/Zsh completion is installed too; start a new shell and press Tab.
```

Each Team Member

```
# 1. Clone your own copy of the code
cd ~
git clone https://github.com/autowarefoundation/autoware.git myname_aw
cd myname_aw
vcs import src < repositories/autoware.repos

# 2. Ask an administrator to register the target
sudo nwctl register myname --src /home/myname/myname_aw/src

# 3. Pre-flight check
nwctl check-env

# 4. Use as needed
nwctl myname planning-sim      # testing
nwctl myname rosbag-replay    # replay
nwctl myname shell             # development
```

CI/CD Pipeline

Push to dev/main branch

↓

ci.yaml (triggered automatically)

- ├ shellcheck static analysis
- └ basic validation (executable, symlink, version, help, install/uninstall)

Push tag v*

↓

release.yaml (triggered automatically)

- ├ reuse ci.yaml
- ├ verify tag matches NWCTL_VERSION in script
- ├ create nwctl-X.Y.Z.tar.gz
- ├ auto-generate changelog
- └ publish GitHub Release (personal + org repos)

Remote Repositories:

Alias	URL	Purpose
origin	git@github.com:LiZheng1997/nwctl.git	Personal repo
org	git@github.com:uwa-rev/nwctl.git	Organization repo

FAQ

Q: rviz2 shows a black screen or crashes

```
xhost +local:docker
echo $DISPLAY # Should be :0 or :1
```

Q: GPU error “could not select device driver”

```
docker run --rm --runtime=nvidia nvidia/cuda:12.0.0-base-ubuntu22.04 nvidia-smi
# If this fails, reinstall nvidia-container-toolkit and restart Docker
```

Q: Multiple users’ ROS topics interfere with each other

Administrator-created development targets are automatically assigned a unique `ROS_DOMAIN_ID` in the range 10-199. Registration, source updates, removal, and `DOMAIN_ID` changes require root. Vehicle/production IDs 0-9 require `sudo nwctl set-domain <target> --domain-id <ID> --production`. `DOMAIN_ID` separates DDS discovery; it is not an authentication or authorization boundary.

Q: colcon build can’t find dependencies

```
source /opt/autoware/setup.bash # source prebuilt layer first
colcon build --packages-select <pkg> --symlink-install
```

Q: Permission error on pointcloud map

```
sudo chmod 644 ~/autoware_map/sample-map-rosbag/pointcloud_map.pcd
# Or run the automated check:
nwctl check-env
```

Q: Are build artifacts preserved after container exit?

Yes. In an installed deployment, build artifacts live in `/var/lib/nwctl/workspaces/<user>/` and persist across container restarts.

Q: How do I switch branches for testing?

Manage your code on the host:

```
cd ~/myname_aw/src/universe/autoware_universe
git checkout feature/new-branch
# Then enter the dev shell and rebuild
nwctl myname shell
```